Network traffic analysis using Wireshark

*BY: **S Aryan Malto***
PRN: SOM25MCA048
DATE: 19-03-2026

The aim of this practical is to understand and perform network traffic analysis using Wireshark — an open-source, graphical packet analyzer. By capturing live network packets and inspecting their contents, students will:

- Gain practical familiarity with Wireshark's interface and core features.
- Understand how data is structured and transmitted across a network in the form of packets.
- Identify and interpret common protocols such as TCP, UDP, HTTP, HTTPS, DNS, and ICMP.
- Apply capture filters and display filters to isolate relevant traffic.
- Analyze real-world packet flows to support cybersecurity investigations, network troubleshooting, and performance monitoring.

What is Network Traffic Analysis?

Network Traffic Analysis (NTA) is the process of capturing, recording, and inspecting data packets that travel across a computer network. Every action that occurs over the internet — loading a webpage, sending an email, streaming a video, or executing a ping command — is broken down into discrete units called packets, and NTA examines those packets to understand what is happening on the network at any given moment.

NTA serves several critical purposes in the domains of cybersecurity, network engineering, and IT operations:

- Security monitoring: Detecting intrusions, malware command-and-control traffic, data exfiltration, and policy violations.
- Troubleshooting: Diagnosing slow connections, failed authentications, and misconfigured services.
- Performance optimization: Identifying bottlenecks, packet loss, and high-latency paths.
- Compliance and forensics: Capturing evidence for incident response and regulatory audits.

What is Wireshark?

Wireshark is a free, open-source network protocol analyzer originally developed by Gerald Combs in 1998 under the name Ethereal. Renamed Wireshark in 2006, it has grown into the world's most widely used packet capture and analysis tool, supported by a large global community. It runs on Windows, macOS, Linux, and other UNIX-based platforms.

Wireshark operates at the data-link layer of the network stack, meaning it can capture raw frames directly from a network interface before the operating system processes them. In promiscuous mode (or monitor mode on wireless adapters), it can even capture traffic not addressed to the host machine, making it extremely powerful for passive network observation.

Key Features of Wireshark

- Live packet capture from wired (Ethernet) and wireless (Wi-Fi) interfaces.
- Deep protocol dissection for hundreds of protocols out of the box.
- Powerful display filter language (BPF-compatible) to narrow down visible packets.
- Color-coded packet listing for quick visual identification of protocol types and anomalies.
- Follow Stream feature to reconstruct TCP, UDP, HTTP, and TLS conversations.
- Export objects (e.g., extract files transferred over HTTP).
- Statistics and I/O graphs for throughput, round-trip time, and flow analysis.
- Saves captures in the industry-standard PCAP and PCAPng formats.

Key Concepts

Packets

A packet is the fundamental unit of data transmission in modern networks. When any application sends data, the operating system's networking stack breaks it into packets, each of which consists of a header and a payload. The header carries routing and control information (source address, destination address, protocol type, sequence numbers, etc.), while the payload carries the actual data. Wireshark captures and displays every layer of this structure.

Network Protocols

TCP (Transmission Control Protocol): A connection-oriented, reliable transport-layer protocol. TCP establishes a connection via a three-way handshake (SYN, SYN-ACK, ACK) and guarantees ordered, error-checked delivery of data. It is used by HTTP, HTTPS, SMTP, and most application protocols that require reliability.

UDP (User Datagram Protocol): A connectionless, lightweight transport-layer protocol with no delivery guarantees. UDP is preferred where speed matters more than reliability — for example, DNS queries, VoIP, video streaming, and online gaming.

HTTP / HTTPS: HyperText Transfer Protocol (and its encrypted variant HTTPS) operates at the application layer and governs how web browsers communicate with web servers. HTTP traffic is plaintext and fully readable in Wireshark; HTTPS is encrypted via TLS and requires a key to decrypt.

DNS (Domain Name System): DNS translates human-readable domain names (e.g., www.example.com) into IP addresses. DNS queries are typically carried over UDP port 53. Analysing DNS traffic in Wireshark can reveal which domains a host is communicating with — valuable information for both troubleshooting and threat hunting.

ICMP (Internet Control Message Protocol): Used for diagnostic messages such as ping (Echo Request / Echo Reply) and Destination Unreachable notifications. ICMP traffic is commonly the first type analysed by students new to Wireshark.

Packet Capture (PCAP)

PCAP stands for Packet Capture, referring both to the act of capturing packets and to the .pcap file format used to store them. Wireshark uses the libpcap library (on Linux/macOS) or Npcap/WinPcap (on Windows) to interface with the network driver and capture raw frames. Saved PCAP files can be shared, analysed offline, or imported into other security tools such as Zeek (Bro), Snort, or Security Onion.

Capture Filters vs Display Filters

Capture Filters are applied before packets are captured. They use Berkeley Packet Filter (BPF) syntax and instruct the capture engine to record only matching packets, reducing file size and processing overhead. Example: **host 192.168.1.1** captures only traffic to/from that IP address.

Display Filters are applied after capture to show only the packets of interest from an already-captured dataset. They are far more expressive than capture filters and support protocol-specific fields. Example: **http. request. Method == "GET"** displays only HTTP GET requests.

Packet Structure

Every packet captured by Wireshark is displayed in three panes:

- Packet List Pane (top): A summary row for each packet — timestamp, source, destination, protocol, length, and a brief description.
- Packet Details Pane (middle): A hierarchical tree showing each protocol layer (Frame, Ethernet, IP, TCP/UDP, Application). Each field can be expanded to reveal its raw value.
- Packet Bytes Pane (bottom): The raw hexadecimal and ASCII representation of the packet's bytes, with the selected field highlighted.

Role of Wireshark in Cybersecurity, Troubleshooting, and Network Monitoring

In cybersecurity, Wireshark is an indispensable tool for:

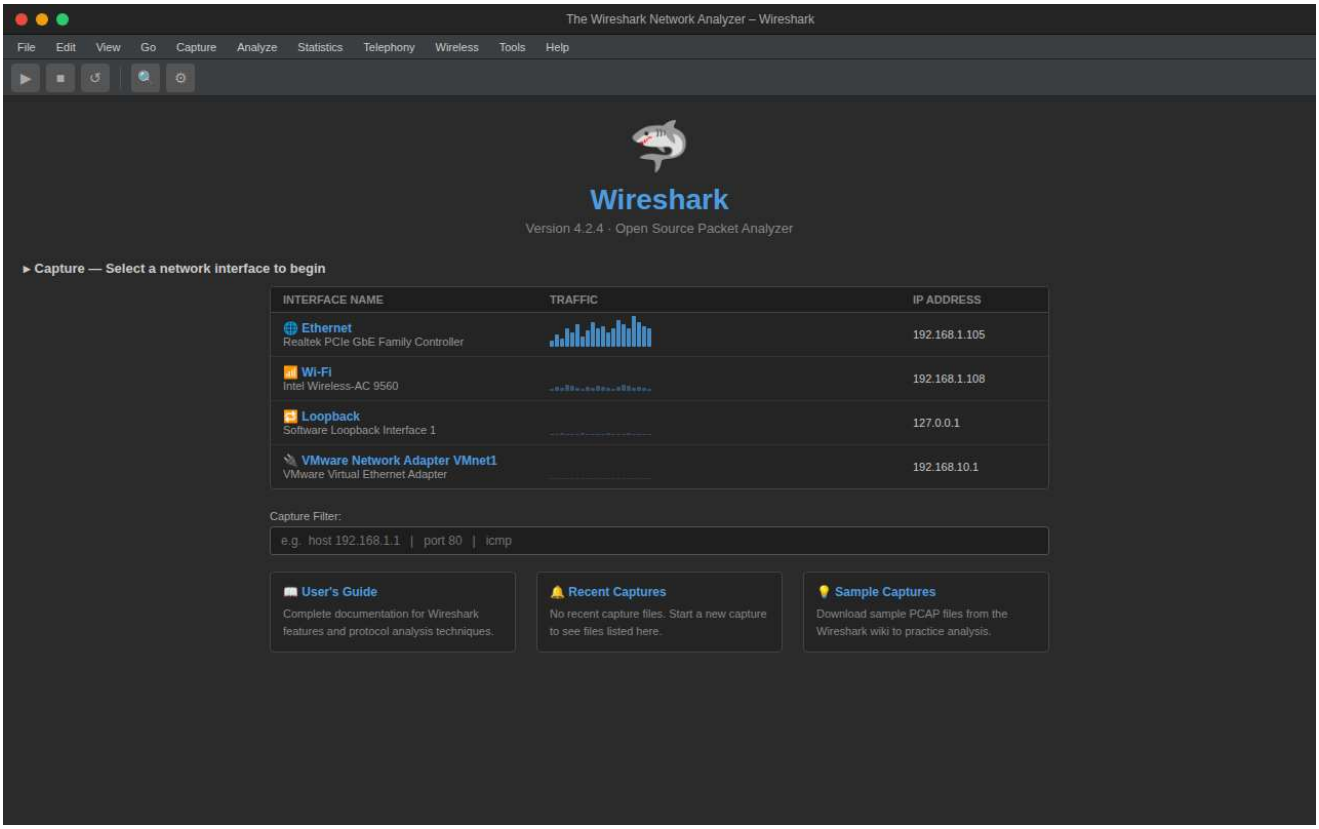
- Incident Response: Analysts replay captured traffic to reconstruct an attack timeline, identify compromised hosts, and determine what data was stolen.
- Malware Analysis: Suspicious processes can be identified by the unusual domains they contact or the anomalous protocols they use.
- Intrusion Detection: Patterns such as ARP spoofing, SYN flood attacks, and port scans are clearly visible in packet captures.
- Credential Harvesting Detection: On legacy unencrypted protocols (e.g., FTP, Telnet, HTTP Basic Auth), Wireshark can reveal credentials transmitted in plaintext — highlighting the need for encryption.

In network troubleshooting, Wireshark allows engineers to verify TCP handshakes, diagnose retransmissions and duplicate ACKs, measure latency between request and response, and pinpoint where in the network path a failure occurs.

Procedure

Step 1: Installation and Setup of Wireshark

1. Visit the official Wireshark website at https://www.wireshark.org and navigate to the Downloads section.
2. Select the appropriate installer for your operating system (Windows, macOS, or Linux).
3. On Windows, run the downloaded .exe installer. During installation, ensure that Npcap is also installed — this is the packet capture driver that Wireshark requires to access the network interface. Accept the default options.
4. On Ubuntu/Debian Linux, install via terminal using the command: sudo apt update && sudo apt install wireshark. When prompted, select Yes to allow non-superusers to capture packets, and add your user to the wireshark group with: sudo usermod -aG wireshark $USER. Log out and back in for the change to take effect.
5. Launch Wireshark. The Welcome Screen displays a list of available network interfaces, each showing a live sparkline graph of current traffic activity.

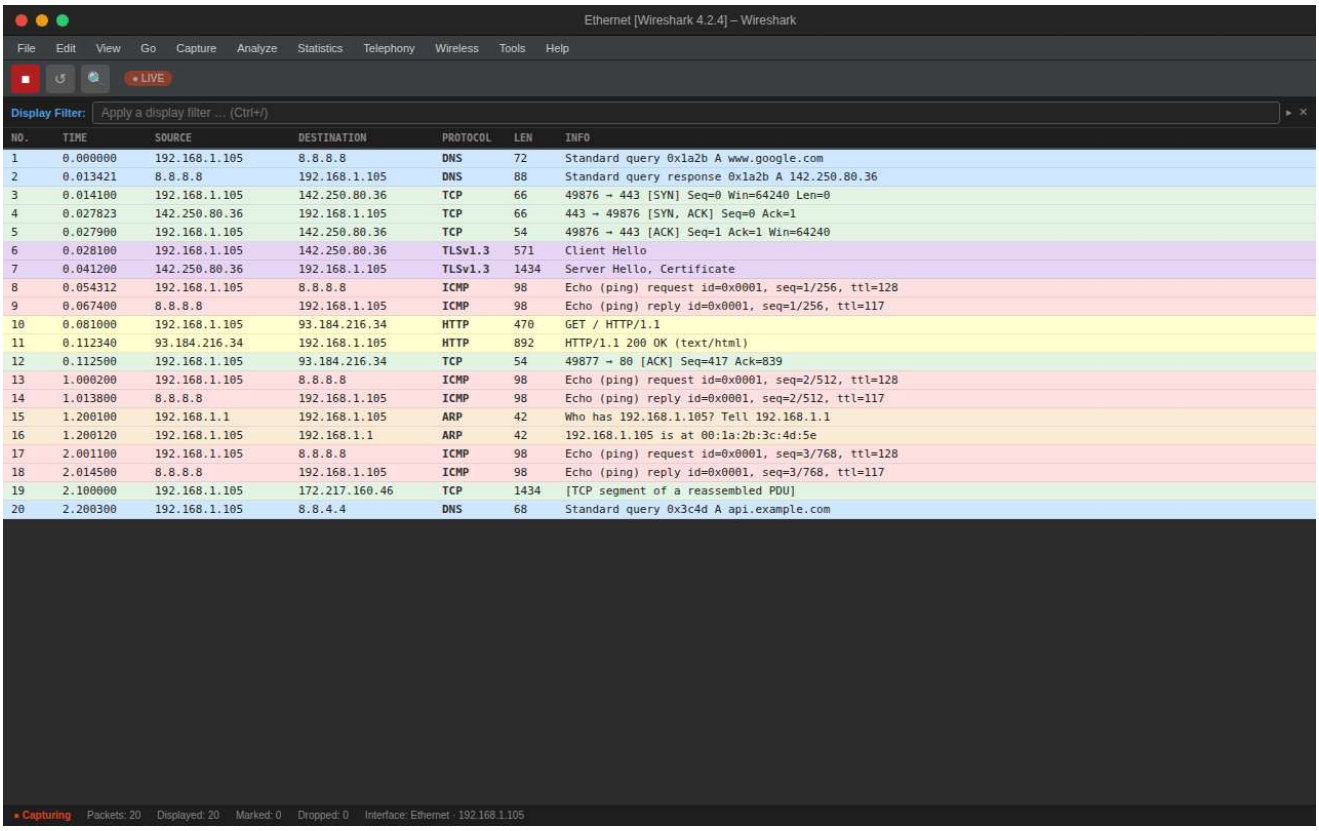


Step 2: Starting a Packet Capture

6. On the Wireshark welcome screen, identify the active network interface. This is usually labelled Ethernet, Wi-Fi, or eth0/wlan0 depending on your operating system. The interface with visible traffic on its sparkline graph is the one currently in use.
7. Double-click the interface name, or select it and click the blue shark-fin icon (Start Capture) in the toolbar. Wireshark immediately begins capturing all packets passing through the interface.

8. Observe the Packet List Pane filling with rows of captured packets. Each row is colour-coded: green rows typically indicate TCP traffic, light blue indicates DNS, dark blue indicates higher-volume TCP, yellow indicates Windows-specific protocols, and red/black indicates errors or malformed packets.

9. To set a Capture Filter before starting (to limit captured data), enter a BPF expression in the Capture Filter bar on the welcome screen. For example, type port 80 to capture only HTTP traffic, or icmp to capture only ping traffic.



Step 3: Generating Network Traffic

To have meaningful data to analyse, generate controlled network traffic during the capture session:

10. Open a web browser and navigate to http://example.com (or any HTTP — not HTTPS — site to see plaintext traffic). This produces HTTP GET and HTTP 200 OK packets.

11. Open a terminal or Command Prompt and execute a ping command:

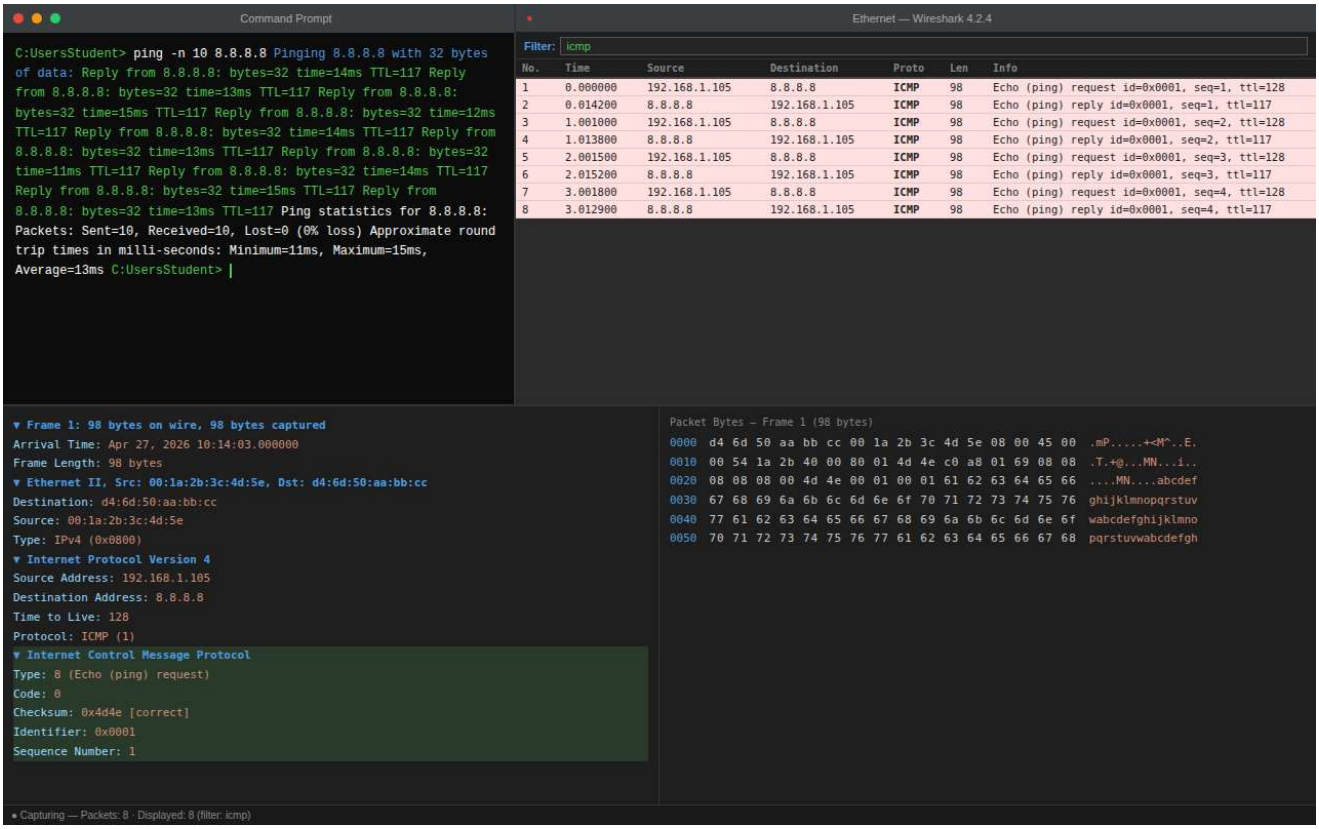
ping google.com (Windows: ping -n 10 google.com)

ping -c 10 google.com (Linux/macOS)

12. Perform a DNS lookup using nslookup or dig:

nslookup example.com

13. After generating traffic, stop the capture by clicking the red Stop button in the toolbar.

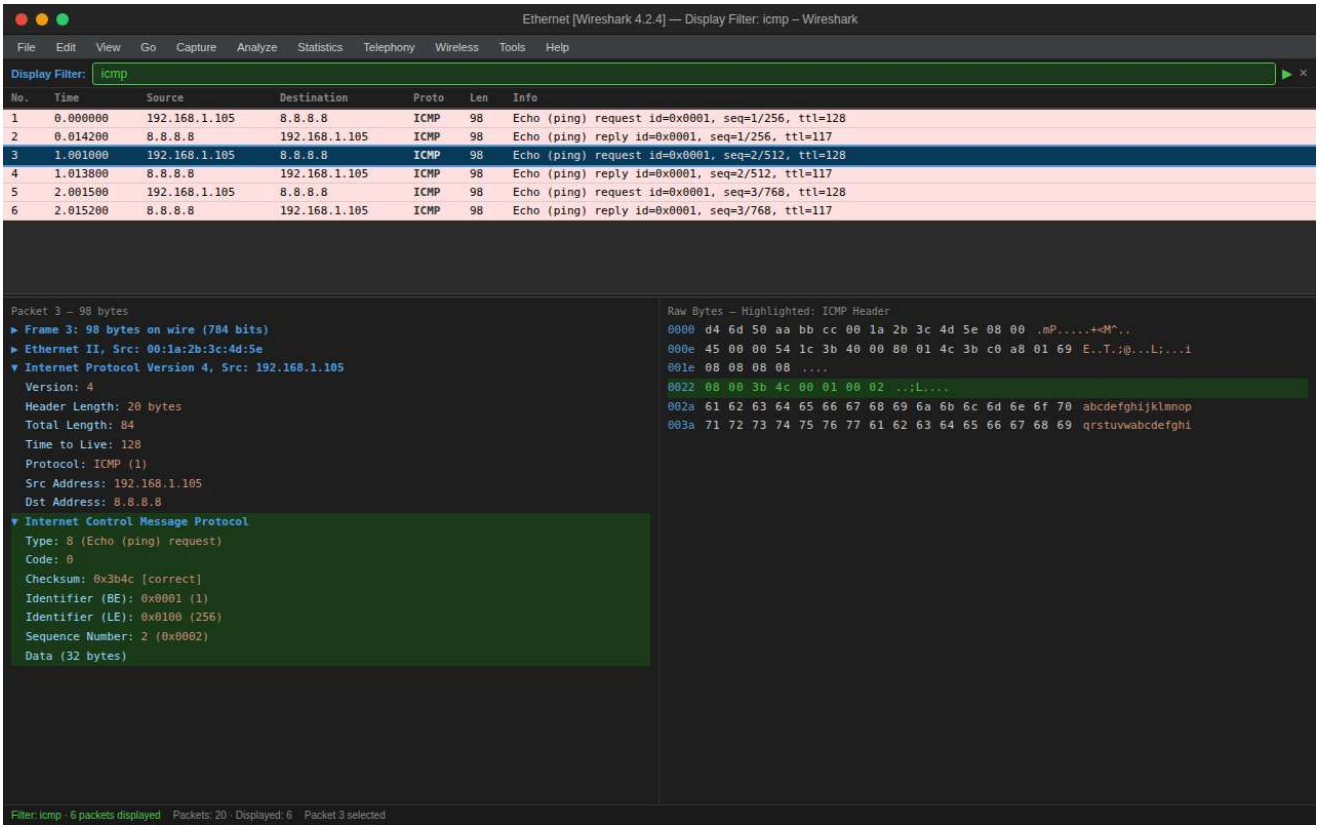


Step 4: Applying Display Filters and Analyzing Packets

With the capture stopped (or still running), use the Display Filter bar at the top of the Wireshark window to isolate specific traffic:

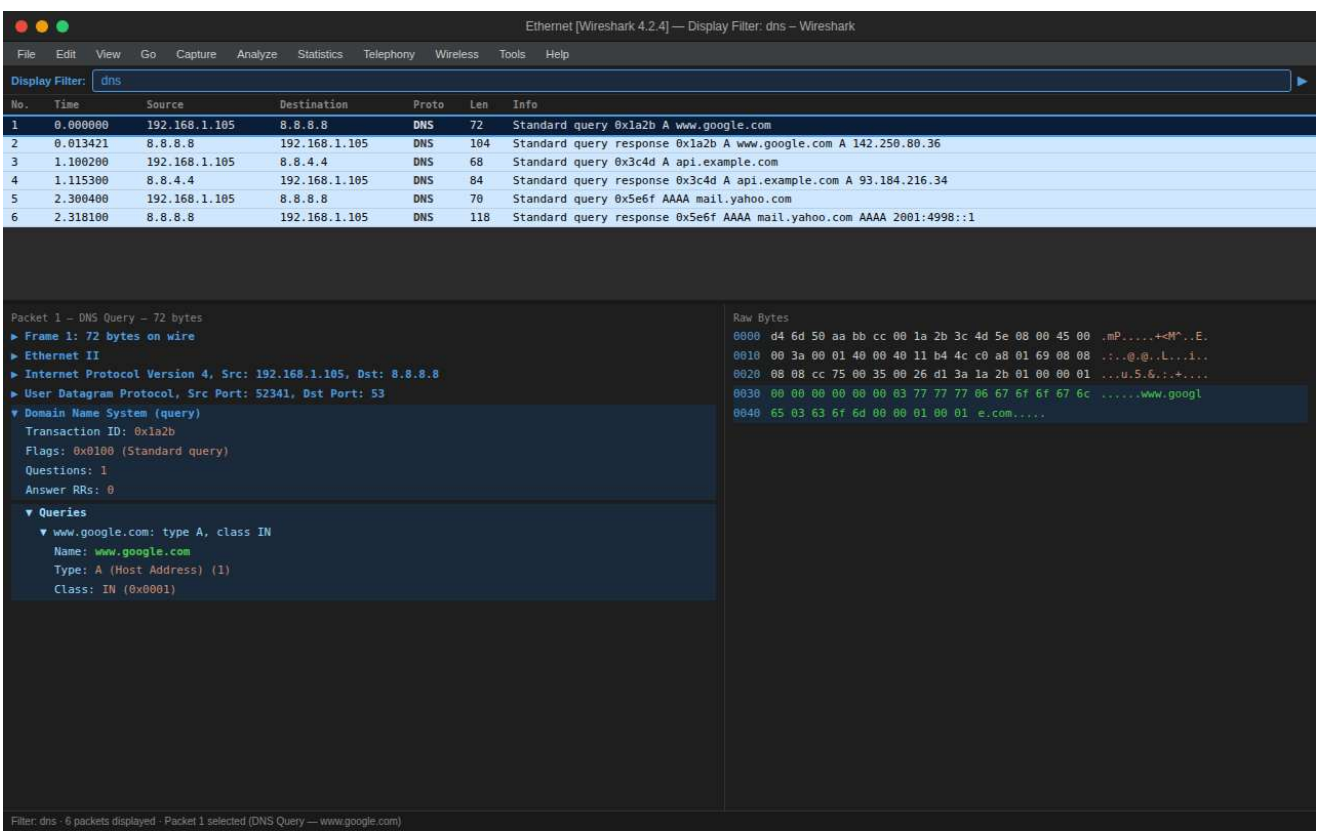
Filtering ICMP (Ping) Traffic

Type icmp in the Display Filter bar and press Enter. Only ICMP Echo Request and Echo Reply packets are shown. Click on any ICMP packet to expand its details in the Packet Details Pane, revealing the Ethernet frame, IP header (source and destination IP), and the ICMP message type and code.



Filtering DNS Traffic

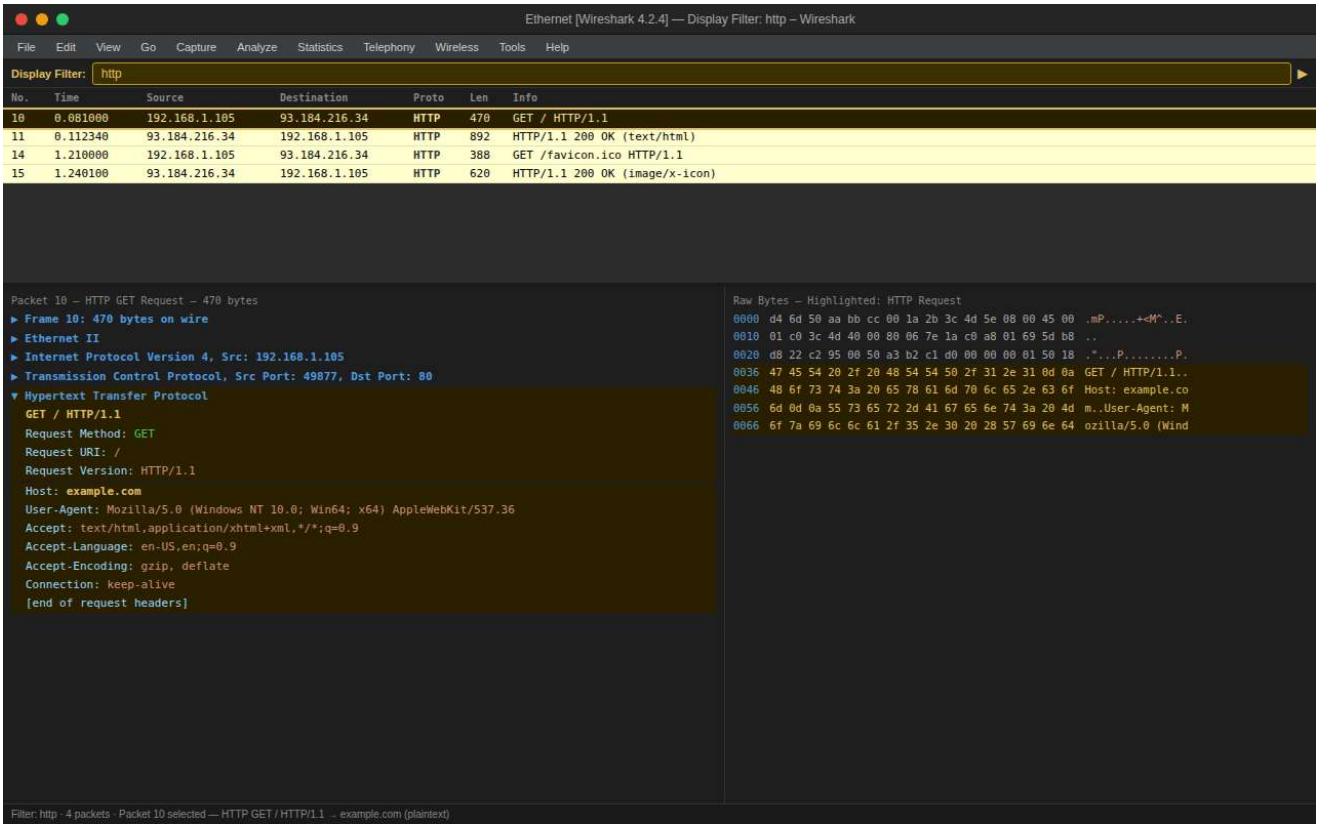
Clear the current filter and type dns in the Display Filter bar. DNS Query and DNS Response packets are listed. Click a DNS Query packet and expand the Domain Name System section in the Packet Details Pane to see the queried domain name. Click the corresponding DNS Response to see the resolved IP address(es) returned by the DNS server.



Filtering HTTP Traffic

Type http in the Display Filter bar. You will see HTTP GET requests (from browser to server) and HTTP 200 OK responses (from server to browser). Click an HTTP GET packet to inspect the request method, requested URL, Host header, and User-Agent string — all of which are visible in plaintext.

> *Note: HTTPS traffic (port 443) is encrypted with TLS. Without the server's private key or a pre-master secret log, the application-layer content of HTTPS packets cannot be read in Wireshark. This demonstrates why HTTPS is recommended over HTTP for all web communication.*



Filtering by IP Address

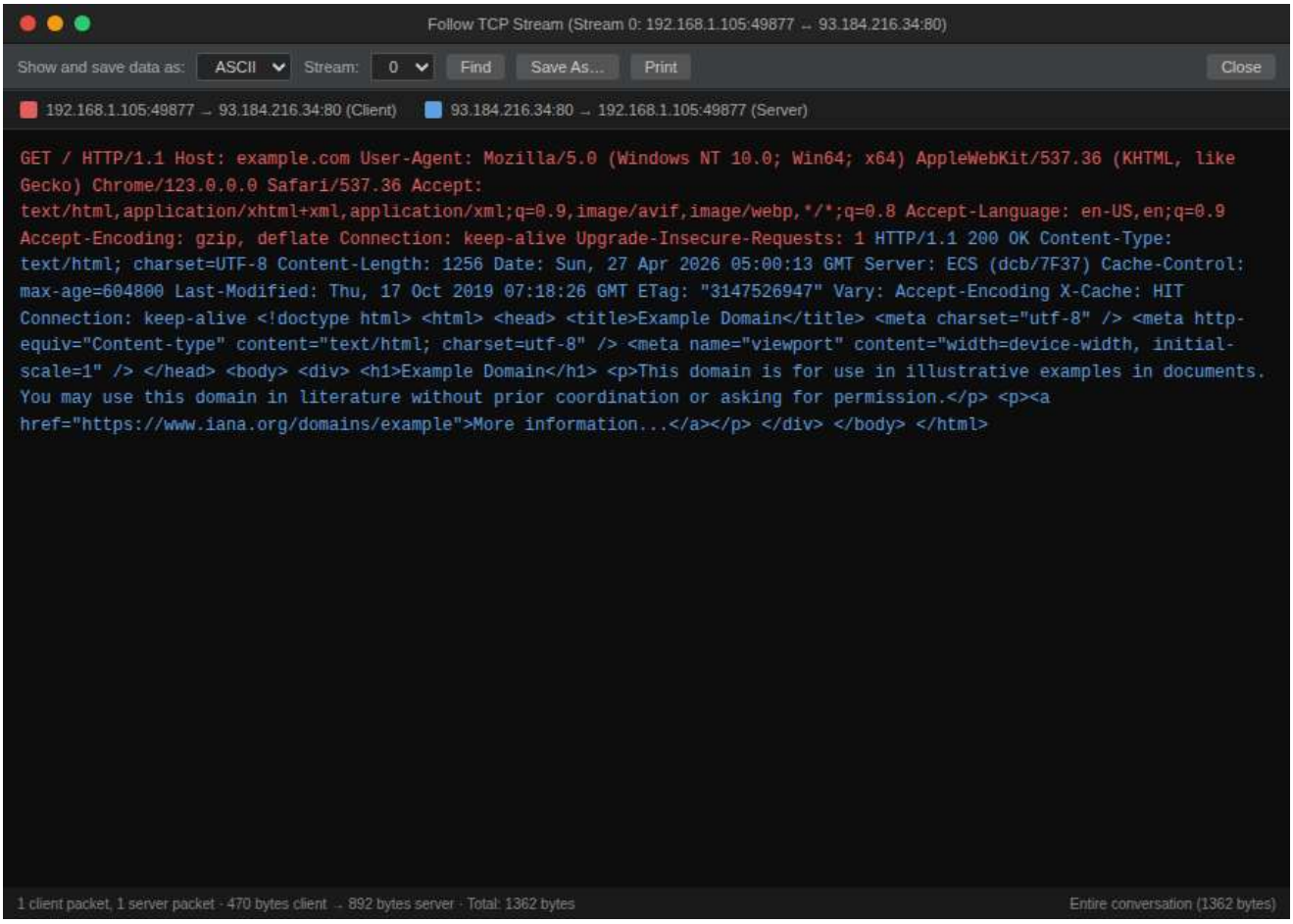
To view all traffic to or from a specific host, use the filter: ip. addr == 8.8.8.8 (replacing the IP with the address of interest). To see only traffic from a specific source, use ip.src == 192.168.1.10. To combine conditions: ip.addr == 8.8.8.8 && tcp.

Step 5: Using the 'Follow TCP Stream' Feature

One of Wireshark's most powerful analysis features is Follow TCP Stream, which reconstructs the complete application-layer conversation between a client and a server from individual TCP packets.

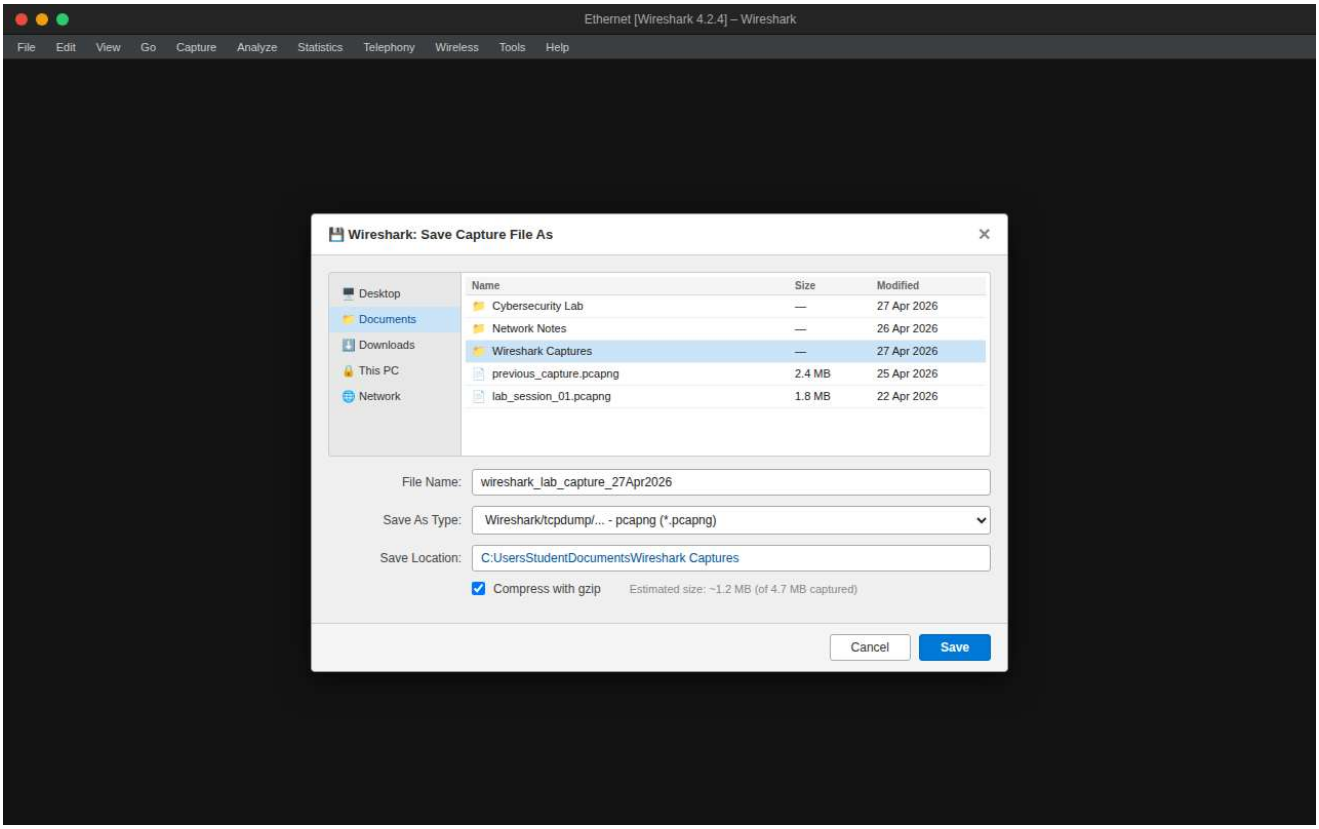
14. Apply the http display filter to show only HTTP traffic.
15. Right-click on any HTTP packet belonging to the session you want to examine.
16. From the context menu, select Follow > TCP Stream.

17. A new window opens showing the entire HTTP conversation in a human-readable format. Client-to-server data is shown in red; server-to-client data is shown in blue.
18. For an HTTP session, you can read the full HTTP request (method, headers, body) and the full HTTP response (status code, response headers, and the page content).
19. Close the TCP Stream window when finished.



Step 6: Saving the Capture File

20. Go to File > Save As (Ctrl + Shift + S).
21. Choose a location and filename, and select the file format. The default format is PCAPng (. pcapng), which supports modern features. For maximum compatibility with other tools, PCAP (.pcap) can be selected.
22. Click Save. The capture file can now be shared, submitted as evidence, or reopened later in Wireshark or another analysis tool such as Zeek or Network Miner.



❖ Conclusion

This practical has demonstrated that Wireshark is a remarkably powerful and accessible tool for network traffic analysis. By capturing live packets, applying targeted filters, and using features such as Follow TCP Stream, it becomes possible to develop a thorough understanding of how data actually moves across a network — not merely how it is described in theory.

Several important conclusions can be drawn from this exercise:

- Visibility into network behaviour: Wireshark makes abstract networking concepts concrete. Seeing a three-way TCP handshake, a DNS query being resolved, or an HTTP GET request and its response in real time is far more instructive than reading about them.
- Security implications of unencrypted protocols: Analyzing HTTP traffic clearly demonstrates the danger of transmitting sensitive information without encryption. Credentials, cookies, and page content are all visible in plaintext — underscoring why HTTPS and encrypted protocols are non-negotiable in modern applications.
- Importance of filters: Practical network captures contain thousands of packets. The ability to use precise capture and display filters is essential for efficient analysis and is a skill that improves with practice.
- Broad applicability: Wireshark is used by network engineers, security analysts, penetration testers, developers debugging APIs, and students learning about protocols. Proficiency with Wireshark is widely regarded as a fundamental skill for anyone working in IT infrastructure or cybersecurity.
- Foundation for advanced topics: These practical lays the groundwork for more advanced topics including intrusion detection, malware traffic analysis, wireless security analysis (using monitor mode), and forensic investigation of PCAP files collected from security tools such as IDS sensors and network taps.